

π_{RL} : Online RL Fine-tuning for Flow-based Vision-Language-Action Models

Kang Chen^{2,6,*} Zhihao Liu^{3,6,*} Tonghe Zhang^{4,*,#} Zhen Guo⁵ Si Xu⁵ Hao Lin⁵
 Hongzhi Zang¹ Xiang Li⁵ Bingwen Wei¹ Jiakai Zhou¹ Quanlu Zhang⁵
 Zhaofei Yu² Guoliang Fan³ Tiejun Huang² Yu Wang^{1†} Chao Yu^{1†}

<https://github.com/RLinf/RLinf> <https://huggingface.co/RLinf>

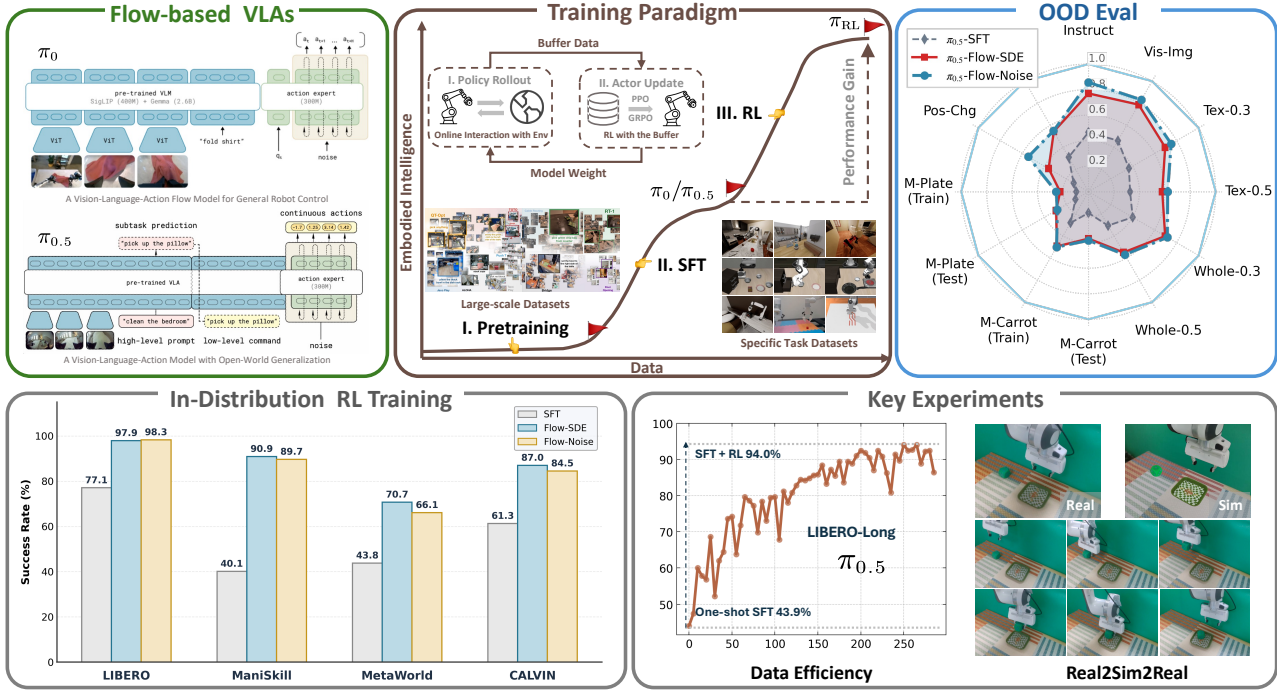


Figure 1. π_{RL} : An online RL framework for flow-based VLAs. Incorporating two solutions, Flow-Noise and Flow-SDE, π_{RL} enhances the performance and generalization of SFT-aligned models across extensive ID benchmarks and OOD settings. Refined with RL, few-shot SFT policies achieve performance comparable to full dataset baselines. Additionally, we facilitate seamless zero-shot sim-to-real transfer by constructing a simulator with 3D Gaussian Splatting as the rendering engine to narrow the visual domain gap.

Abstract

Vision-Language-Action (VLA) models enable robots to understand and perform complex tasks from multimodal input. Although recent work explores using reinforcement learning (RL) to automate the laborious data collection process in scaling supervised fine-tuning (SFT), applying RL to large-scale flow-based VLAs (e.g., π_0 ,

$\pi_{0.5}$) remains challenging due to intractable action log-likelihoods raised from flow matching. We address this challenge with π_{RL} , featuring two technical approaches: (1) **Flow-Noise** models the denoising process as a discrete-time MDP with a learnable noise network for exact log-likelihood computation. (2) **Flow-SDE** integrates denoising with agent-environment interaction, formulating a two-layer MDP that employs ODE-to-SDE conversion for efficient RL exploration. We evaluate π_{RL} across various benchmarks, with experiments demonstrating that RL yields significant performance improvements in both in-distribution and out-of-distribution settings.

¹Tsinghua University ²Peking University ³Institute of Automation, Chinese Academy of Sciences ⁴Carnegie Mellon University ⁵Infinigence AI ⁶Zhongguancun Academy # Work done at Tsinghua University * Equal Contributions . Correspondence to: Yu Wang <yu-wang@tsinghua.edu.cn>, Chao Yu <zoe-yuchao@gmail.com>.

1. Introduction

Vision-Language-Action (VLA) models (Din et al., 2025) have emerged as a leading solution for general-purpose robots, effectively bridging the gap between high-level multimodal reasoning and low-level physical control (Firoozi et al., 2025). Conditioned on sensor inputs and language commands, VLAs (Team et al., 2024; Kim et al., 2024; Black et al., 2024; Intelligence et al., 2025) can translate abstract instructions into executable robotic actions, thereby enabling intuitive and flexible human-robot interaction.

The training methodology for VLAs follows the standard pre-training and supervised fine-tuning (SFT) paradigm as shown in Fig. 1. Building on the pretrained Vision-Language Model (VLM) (Touvron et al., 2023; Beyer et al., 2024), VLAs are fine-tuned on large-scale, heterogeneous human demonstration datasets (O’Neill et al., 2024; Khazatsky et al., 2024), followed by SFT on the target task to align their capabilities with the specific embodiment and environment. However, reliance on SFT introduces a critical challenge: curating large-scale, high-quality expert trajectories is both laborious and costly (Din et al., 2025). Besides, models obtained via SFT tend to overfit to expert demonstrations (Fei et al., 2025), with their performance fundamentally constrained by the quality of expert demonstrations.

Recent efforts (Zang et al., 2025; Li et al., 2025a; Tan et al., 2025; Liu et al., 2025a) have explored expanding the VLA training process with reinforcement learning (RL), establishing a pre-training, SFT, and RL paradigm as shown in Fig. 1, allowing VLAs to improve their performance beyond expert demonstrations through environmental interaction and develop more generalizable policies.

However, these RL advances have been largely confined to autoregressive VLAs, featuring OpenVLA (Kim et al., 2024) and OpenVLA-OFT (Kim et al., 2025), which employ discrete action decoders that generate output in an autoregressive or parallel fashion. This stands in stark contrast to flow-based VLAs, exemplified by the π series models, which generate actions through iterative refinement in flow matching (Lipman et al., 2022), offering the advantages of generating action chunks in high-frequency and performing highly dexterous tasks (Black et al., 2024). Consequently, previous VLA-RL algorithms are incompatible with flow-based VLAs, and the fundamental challenge lies in how to characterize a logarithmic likelihood (Hutchinson, 1989; Chen et al., 2018) for the executed actions.

In this paper, we introduce π_{RL} , a framework designed for fine-tuning flow-based VLAs with online RL algorithms. To address the intractable log-likelihood estimation problem in flow matching, we propose two solutions. **Flow-Noise** integrates a learnable noise network into the denoising process and models this stage as a discrete-time Markov decision

process (MDP) for exact log-likelihood estimation. **Flow-SDE** converts the ordinary differential equation (ODE) denoising process into a stochastic differential equation (SDE) while maintaining equivalent marginal distributions for exploration, and builds a two-layer MDP that couples the denoising process with policy-environment interaction. Given the formulated MDP and the exact log-likelihood computation, π_{RL} undergoes further optimization via the proximal policy optimization (PPO) (Schulman et al., 2017).

We conduct extensive experiments on various benchmarks to evaluate the effectiveness of π_{RL} on π_0 (Black et al., 2024) and $\pi_{0.5}$ (Intelligence et al., 2025) models. Across all benchmarks, the proposed framework consistently yields substantial performance gains over SFT baselines. Furthermore, out of distribution evaluations confirm that our model yields genuine policy enhancement rather than narrow overfitting on the target environment.

To sum up, our contributions are:

- **RL for flow-based VLAs.** We introduce π_{RL} , an online RL fine-tuning framework with Flow-Noise and Flow-SDE formulations for flow-based VLAs.
- **Superior Performance.** We demonstrate significant performance improvements and enhanced generalization of π_{RL} across various benchmarks.
- **Comprehensive Ablation.** We conduct thorough ablation studies, offering empirical insights to guide future RL research on flow-based VLAs.
- **Open-source Code and Models.** We release all codes to ensure reproducibility, hoping that our study helps to advance further research in this field.

2. Related Work

2.1. Vision-Language-Action Models

VLA models have recently achieved remarkable progress in robotics by integrating multimodal inputs to enable unified perception, reasoning, and control. This development has led to a series of architectures, including Octo (Team et al., 2024), RT (Brohan et al., 2022), OpenVLA, OpenVLA-OFT, π_0 , $\pi_{0.5}$, and GR00T (Bjorck et al., 2025).

2.2. Online RL Fine-tuning for VLA Models

Recent research has increasingly focused on enhancing the performance and generalization of VLAs with online RL. For example, SimpleVLA-RL (Li et al., 2025a), building on the OpenVLA-OFT and GRPO, demonstrated that RL can improve long-horizon planning of VLA models under data scarcity. RL4VLA (Liu et al., 2025a) empirically evaluated PPO, GRPO, and direct preference optimization

(DPO) (Rafailov et al., 2023) with stage-based sparse rewards. RLinf-VLA (Yu et al., 2025; Zang et al., 2025) provides a unified and efficient framework for scalable RL training of VLA models. These works demonstrate the effectiveness of RL fine-tuning VLA models.

2.3. RL Fine-tuning for Flow Models

Integrating RL with flow models is a promising way to transcend the limitations of imitation learning. To this end, Flow-GRPO (Liu et al., 2025b) converts the deterministic ODE into an equivalent SDE to enable stochasticity exploration, a foundation upon which subsequent works like Mix-GRPO (Li et al., 2025b) and TempFlow-GRPO (He et al., 2025) further accelerate training through hybrid ODE-SDE rollouts. ReinFlow (Zhang et al., 2025) injects learnable noise into the flow path and transforms it into a discrete-time Markov process with a tractable likelihood for stable policy gradient updates. Flow policy optimization (FPO) (McAllister et al., 2025) reframes policy optimization as maximizing the advantage-weighted ratio of the conditional flow matching loss.

3. Preliminary

3.1. Problem Formulation

We formulate the task as an MDP, defined by a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P_0, P_{\text{ENV}}, R_{\text{ENV}}, \gamma)$. The state $s_t \in \mathcal{S}$ is defined as the robot observation $\mathbf{o}_t$ and P_0 denotes the initial state distribution. Given the state, the flow policy predicts an action $a_t \sim \pi(\cdot|s_t) \in \mathcal{A}$, resulting in the state transition $s_{t+1} \sim P_{\text{ENV}}(\cdot|s_t, a_t)$ and a reward $R_{\text{ENV}}(s_t, a_t)$. The objective is to learn a policy π_θ that maximizes the expected γ -discounted return over a horizon of $T + 1$:

$$\mathcal{J}(\pi_\theta) = \mathbb{E}_{\pi_\theta, P_0} \left[\sum_{t=0}^T \gamma^t R_{\text{ENV}}(s_t, a_t) \right]. \quad (1)$$

With the policy gradient surrogate (Williams, 1992), the gradient of the return expectation can be approximated from sampled trajectories:

$$\nabla_\theta \mathcal{J}(\pi_\theta) = \mathbb{E}_{\pi_\theta, P_0} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) A(s_t, a_t) \right]. \quad (2)$$

The advantage function, $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, measures the relative merit of the action value $Q(s_t, a_t)$ over the state value $V(s_t)$, providing a low-variance signal for the policy update.

3.2. Flow-based Vision-Language-Action Model

A flow-based VLA model π_θ is designed to map the observation $\mathbf{o}_t$ comprising RGB images, language tokens, and

robot proprioception to a sequence of H future actions $\mathbf{A}_t = [a_{t,0}, \dots, a_{t,H-1}]$, formulated as $p(\mathbf{A}_t|\mathbf{o}_t)$. Within the model, the VLM extracts features from the visual and language inputs, while the flow matching expert is tasked with generating the actions. Specifically, the model learns a conditional vector field $\mathbf{v}_\theta$ that transforms a standard Gaussian noise distribution into the target action $\mathbf{A}_t$. This is achieved by minimizing the Conditional Flow Matching (CFM) loss, which aligns the predicted vector field $\mathbf{v}_\theta$ with the ground-truth vector field $\mathbf{u}$:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{\tau, p(\mathbf{A}_t, \mathbf{o}_t), q(\mathbf{A}_t^\tau|\mathbf{A}_t)} \left[\|\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) - \mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t)\|_2^2 \right]. \quad (3)$$

Here, the conditional probability path $q(\mathbf{A}_t^\tau|\mathbf{A}_t)$ generates a noisy action¹ $\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau)\epsilon$ from an action $\mathbf{A}_t$, random noise $\epsilon \sim \mathcal{N}(0, I)$, and a continuous time $\tau \in [0, 1]$ in flow matching. For this specific path, the corresponding ground-truth vector field is defined as $\mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t) = \mathbf{A}_t - \epsilon$.

During the inference, the action sequence is generated by first sampling a noise vector $\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$, which is further iteratively refined by integrating the learned vector field $\mathbf{v}_\theta$ over a fixed number of steps based on the forward Euler method: $\mathbf{A}_t^{\tau+\delta} = \mathbf{A}_t^\tau + \mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) \cdot \delta$.

4. Methodology

Existing VLA-RL approaches leverage base models such as OpenVLA for discrete actions and OpenVLA-OFT for continuous actions. To compute the action log-likelihood $\log \pi_\theta(a_t|s_t)$, discrete models (Liu et al., 2025a) apply softmax to the output logits, while continuous models (Li et al., 2025a) treat the action as a Gaussian distribution, employing a prediction head to estimate the variance. As for the flow-based VLAs, directly computing the exact likelihood (Hutchinson, 1989) is inaccurate with few denoising steps. Moreover, the deterministic nature of its ODE sampling process precludes exploration, making its implementation within RL non-trivial. To this end, we propose Flow-Noise and Flow-SDE, two technical approaches that make flow-based VLAs amenable to RL.

4.1. Flow-Noise

Inspired by Reinflow (Zhang et al., 2025), we incorporate a learnable noise network into the flow matching denoising process and solve the problem within the standard one-layer MDP framework detailed in Sec. 3.1. By modeling the denoising stage as a discrete MDP, we can directly compute the log-likelihood of the denoised sequence, enabling equivalent policy optimization via RL.

¹ $\mathbf{A}_t^\tau$ incorporates two temporal indices, t denotes the discrete time step for environment interaction and τ represents the continuous time variable in flow matching.

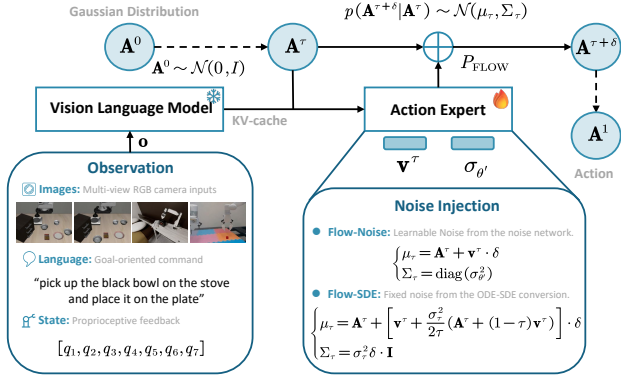


Figure 2. Illustration for the noise injection in π_{RL} .

4.1.1. STOCHASTICITY INJECTION

In Flow-Noise, we parameterize the noise schedule with a neural network, allowing the magnitude of the injected noise to be learned dynamically during training for greater flexibility, as shown in Fig. 2. We focus on the generation process within a single environment timestep t . For notational simplicity, we omit the time subscript t , *e.g.*, writing $\mathbf{A}^\tau$, and denote the predicted velocity $\mathbf{v}_\theta(\mathbf{A}^\tau, \mathbf{o})$ as $\mathbf{v}^\tau$.

The step transition during the denoising process is modeled as a Gaussian distribution $p(\mathbf{A}^{\tau+\delta}|\mathbf{A}^\tau) \sim \mathcal{N}(\mu_\tau, \Sigma_\tau)$, where the mean is determined by the forward Euler update of the original ODE and the variance is controlled by the learnable noise network θ' :

$$\begin{cases} \mu_\tau = \mathbf{A}^\tau + \mathbf{v}^\tau \cdot \delta \\ \Sigma_\tau = \text{diag}(\sigma_{\theta'}^2) \end{cases} \quad (4)$$

Here, $\sigma_{\theta'}(\cdot)$ is the standard deviation learned from the noise injection network, conditioned on the action $\mathbf{A}^\tau$, and the observation $\mathbf{o}$. The noise network is trained jointly with the velocity but discarded after fine-tuning, leaving a deterministic policy for inference.

4.1.2. LOG-LIKELIHOOD ESTIMATION

The primary challenge in applying policy gradient methods to flow-based VLAs stems from the intractable log-likelihood of the final executed action. In Flow-Noise, we address it by substituting the gradient of the joint log-likelihood of the entire denoising process into the policy optimization objective in Eq. (2), which is theoretically grounded in Reinfo (Zhang et al., 2025).

The inference process for action generation is discretized into K uniform steps, which defines a sequence of time points $\{\tau_0, \tau_1, \dots, \tau_K\}$. With the step interval defined as $\delta = 1/K$, the discrete timestep at the k -th point is $\tau_k = k \cdot \delta$, starting from $\tau_0 = 0$ and culminating at $\tau_K = 1$. Given the observation $\mathbf{o}$, the exact and tractable log probability for the

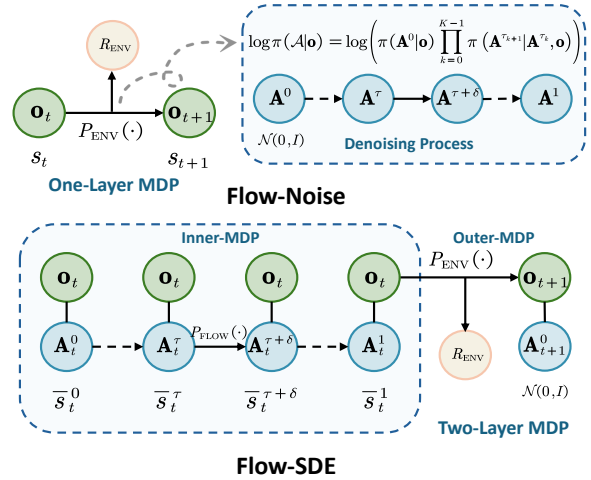


Figure 3. Illustration of the MDP formulations in π_{RL} .

entire denoising sequence $\mathcal{A} = (\mathbf{A}^0, \dots, \mathbf{A}^1)$ is depicted in Fig. 3 and formulated as:

$$\log \pi(\mathcal{A}|\mathbf{o}) = \log \left(\pi(\mathbf{A}^0|\mathbf{o}) \prod_{k=0}^{K-1} \pi(\mathbf{A}^{\tau_{k+1}}|\mathbf{A}^{\tau_k}, \mathbf{o}) \right). \quad (5)$$

Building on this, we can treat flow-based policy optimization within a standard MDP framework.

4.2. Flow-SDE

Inspired by Flow-GRPO (Liu et al., 2025b), we enhance stochastic exploration by converting the denoising process from ODE into an SDE formulation. We further construct a two-layer MDP to couple the denoising process with the policy-environment interaction following DPPO (Ren et al., 2024), while leveraging the hybrid ODE-SDE sampling technique to accelerate the training process.

4.2.1. STOCHASTICITY INJECTION

In Flow-SDE, we convert the deterministic ODE into an equivalent SDE that preserves the marginal probability density of the generated actions, as shown in Fig. 2.

The deterministic ODE sampling trajectory of the flow matching, especially the Rectified Flow (Liu et al., 2022), is described by the forward Euler method:

$$d\mathbf{A}^\tau = \mathbf{v}^\tau d\tau. \quad (6)$$

Building on the connection between the *probability flow* ODE and SDE (Song et al., 2020), we can transform the deterministic ODE in Eq. (6) into an equivalent SDE, with a drift term that corrects the original velocity and a diffusion

term that introduces noise:

$$d\mathbf{A}^\tau = \underbrace{\left(\mathbf{v}^\tau - \frac{1}{2} g^2(\tau) \nabla \log q_\tau(\mathbf{A}^\tau) \right) d\tau}_{\text{Drift Term}} + \underbrace{g(\tau) d\mathbf{w}}_{\text{Diffusion Term}}, \quad (7)$$

where $g(\tau)$ is a scalar function controlling the noise schedule, $\nabla \log q_\tau(\mathbf{A}^\tau)$ is the score function of the marginal distribution q_τ and $d\mathbf{w}$ denotes a Wiener process.

As established in Flow-GRPO, the score function and the velocity field are critically linked by $\nabla \log q_\tau(\mathbf{A}^\tau) = -\frac{\mathbf{A}^\tau}{\tau} - \frac{1-\tau}{\tau} \mathbf{v}^\tau$. By substituting the score function with the velocity field in Eq. (7) and setting the noise schedule $g(\tau)$ to $\sigma_\tau = a\sqrt{\frac{\tau}{1-\tau}}$ with a controlling the noise level, we derive the final SDE formulation for the flow-matching sampler:

$$d\mathbf{A}^\tau = \left[\mathbf{v}^\tau + \frac{\sigma_\tau^2}{2\tau} (\mathbf{A}^\tau + (1-\tau)\mathbf{v}^\tau) \right] d\tau + \sigma_\tau d\mathbf{w}_\tau. \quad (8)$$

Discretizing this SDE reveals that the transition probability $p(\mathbf{A}^{\tau+\delta}|\mathbf{A}^\tau) \sim \mathcal{N}(\mu_\tau, \Sigma_\tau)$ is an isotropic Gaussian distribution, with the mean and variance formulated as:

$$\begin{cases} \mu_\tau = \mathbf{A}^\tau + \left[\mathbf{v}^\tau + \frac{\sigma_\tau^2}{2\tau} (\mathbf{A}^\tau + (1-\tau)\mathbf{v}^\tau) \right] \cdot \delta \\ \Sigma_\tau = \sigma_\tau^2 \delta \cdot \mathbf{I} \end{cases} \quad (9)$$

4.2.2. MDP FORMULATION

We couple the denoising process of the flow matching with environmental interaction in Flow-SDE. Specifically, we embed the inner MDP defined during the denoising process into the high-level, outer-loop MDP with the environment $\mathcal{M}_{\text{ENV}}$ in Sec. 3.1, formulating a two-layer MDP as shown in Fig. 3, with components defined with respect to the environment time t and denoising time τ .

- **State** $\bar{s}_t^\tau = (\mathbf{o}_t, \mathbf{A}_t^\tau)$ is the tuple of the observation $\mathbf{o}_t$ and the action state $\mathbf{A}_t^\tau$.
- **Action** $\bar{a}_t^\tau$ is defined as the next sampled denoised action in the inner-loop and the executed action for the outer loop:

$$\bar{a}_t^\tau = \begin{cases} \mathbf{A}_t^{\tau+\delta} & \text{if } \tau < 1 \\ \mathbf{A}_t^1 & \text{if } \tau = 1 \end{cases}, \quad (10)$$

where $\mathbf{A}_t^{\tau+\delta} = \mu_\tau + \sigma_\tau \sqrt{\delta} \cdot \epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ is the randomly sampled noise.

- **Transition** $\bar{P}(\bar{s}_{t'}^\tau | \bar{s}_t^\tau, \bar{a}_t^\tau)$ defines how the state evolves, formulated as:

$$\bar{s}_{t'}^\tau = \begin{cases} (\mathbf{o}_t, \bar{a}_t^\tau) & \text{if } \tau < 1 \\ (\mathbf{o}_{t+1}, \mathbf{A}_{t+1}^0) & \text{if } \tau = 1 \end{cases}. \quad (11)$$

For $\tau < 1$, the inner loop transition $P_{\text{FLOW}}(\cdot)$ occurs between different denoised action states, where the observation $\mathbf{o}_t$ remains fixed and the next action state is set by $\bar{a}_t^\tau = \mathbf{A}_t^{\tau+\delta}$.

For $\tau = 1$, the final action $\bar{a}_t^\tau = \mathbf{A}_t^1$ interacts with the outer-loop environment, resulting in a new observation $\mathbf{o}_{t+1}$ according to the environment dynamics $P_{\text{ENV}}(\cdot)$. Concurrently, the action state is reset from a standard normal distribution $\mathbf{A}_{t+1}^0 \sim \mathcal{N}(0, \mathbf{I})$.

- **Reward** $\bar{R}(\bar{s}_t^\tau, \bar{a}_t^\tau)$ is granted only upon completion of the denoising process and interaction with the environment:

$$\bar{R}(\bar{s}_t^\tau, \bar{a}_t^\tau) = \begin{cases} 0 & \text{if } \tau < 1 \\ R_{\text{ENV}}(\mathbf{o}_t, \mathbf{A}_t^1) & \text{if } \tau = 1 \end{cases}. \quad (12)$$

Within the two-layer MDP framework, the problem of estimating the action log-likelihood $\log \pi(a_t|s_t)$ is transformed into estimating $\log \pi(\bar{a}_t^\tau | \bar{s}_t^\tau)$, which is straightforward to compute due to the Gaussian nature of the transitions.

4.2.3. HYBRID ODE-SDE SAMPLING

The two-layer MDP formulation significantly extends the horizon, increasing training difficulty and computational cost. To mitigate this, we adopt a mixed ODE-SDE rollout strategy (Li et al., 2025b; He et al., 2025). At each step t , we randomly sample a denoising time τ_t for the stochastic SDE exploration, while treating all remaining denoising steps as deterministic ODE updates. Specifically, the policy acts on the state $\bar{s}_t^{\tau_t} = (\mathbf{o}_t, \mathbf{A}_t^{\tau_t})$; subsequently, an environment wrapper executes the remaining ODE steps and the environment transition, ultimately yielding the next state $\bar{s}_{t+1}^{\tau_{t+1}} = (\mathbf{o}_{t+1}, \mathbf{A}_{t+1}^{\tau_{t+1}})$ at a newly sampled time τ_{t+1} . This formulation effectively shortens the MDP horizon while maintaining theoretical consistency with the original two-layer framework.

4.3. Policy Optimization

4.3.1. ALGORITHM

Given the formulated flow policy MDP, our objective is to learn the optimal parameters θ^* for the policy π_θ that maximizes the expected discounted return $\mathcal{J}(\pi_\theta)$. To this end, we apply the widely adopted policy gradient algorithm PPO to optimize the policy.

π -series models (Black et al., 2024; Intelligence et al., 2025) adopt a chunk-based approach for action generation. Specifically, the policy outputs an entire sequence of H future actions $\mathbf{A}_t = [a_{t,0}, \dots, a_{t,H-1}]$ in response to each observation. In this approach, we treat the entire sequence as a single macro-step and define its corresponding reward $R_t = \sum_{j=0}^{H-1} r_{t,j}$ as the sum of the per-step rewards $r_{t,j}$,

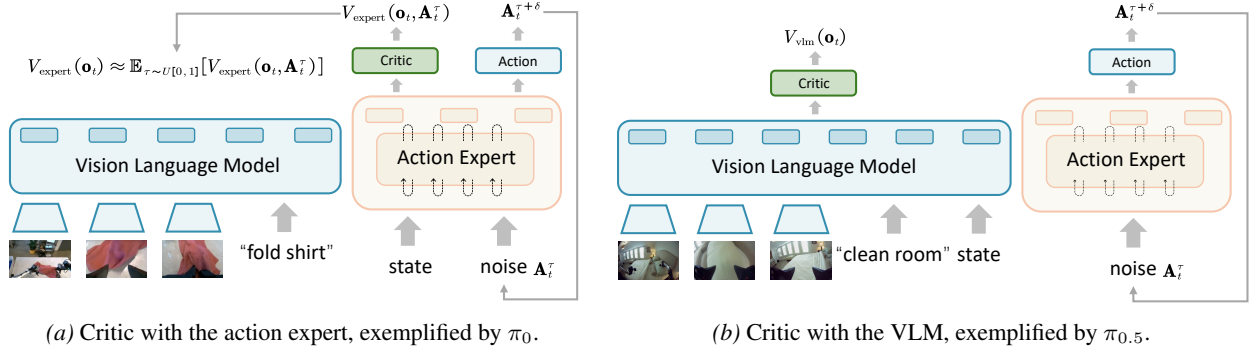


Figure 4. Illustration of the two critic placement configurations.

referred to as the chunk-level formulation in RLinf-VLA (Zang et al., 2025).

To effectively guide policy updates, PPO employs Generalized Advantage Estimation (GAE) (Schulman et al., 2015) to compute a low-variance estimate of the advantage, estimated as:

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma\lambda)^k \mathcal{T}_{t+k}, \quad (13)$$

where the TD-error is $\mathcal{T}_t = R_t + \gamma V(s_{t+1}) - V(s_t)$. Here, $V(\cdot)$ is the state-value function derived from the critic network, γ is the discount factor, and λ is the parameter that balances the trade-off between bias and variance in the advantage estimate.

PPO constrains policy updates to a small trust region to prevent large, destabilizing updates, with the objective function:

$$\mathcal{J}(\pi_\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (14)$$

where the clip function, governed by a hyperparameter ϵ , restricts the ratio $\rho_t(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$ to ensure training stability.

Here, the probability ratio $\rho_t(\theta)$ between the updated and old policies takes the form of either:

$$\rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad \text{or} \quad \rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(\bar{a}_t^\tau | \bar{s}_t^\tau)}{\pi_{\theta_{\text{old}}}(\bar{a}_t^\tau | \bar{s}_t^\tau)}. \quad (15)$$

4.3.2. CRITIC DESIGN

Following VLA-PPO works (Zang et al., 2025; Liu et al., 2025a), we employ a shared actor-critic architecture for memory-efficient value prediction as shown in Fig. 4. However, the two flow-based VLAs process the proprioceptive state differently: in π_0 , the state is fed into the action expert model, whereas in $\pi_{0.5}$, it is merged with prompt embeddings within the VLM.

To this end, for the $\pi_{0.5}$ variant, we attach the critic network directly to the VLM output, providing the value estimate

$V_{\text{vlm}}(\mathbf{o}_t)$ conditioned on the integrated image, language, and state inputs. Conversely, for the π_0 variant, achieving the value prediction is non-trivial due to the coupled input structure, where the action expert requires both the noisy action $\mathbf{A}_t^\tau$ and the state. To this end, we approximate $V_{\text{expert}}(\mathbf{o}_t)$ by averaging the value estimates across the entire denoising trajectory, formulated as:

$$V_{\text{expert}}(\mathbf{o}_t) \approx \mathbb{E}_{\tau \sim U[0,1]}[V_{\text{expert}}(\mathbf{o}_t, \mathbf{A}_t^\tau)]. \quad (16)$$

5. Experimental Results

5.1. Setup

Benchmarks. We perform experiments on four widely-adopted robot manipulation benchmarks: **LIBERO** (Liu et al., 2023), **ManiSkill** (Tao et al., 2024), **MetaWorld** (McLean et al., 2025) and **CALVIN** (Mees et al., 2022).

Flow-based VLAs. We conduct our primary experiments based on π_0 and $\pi_{0.5}$ models. Additionally, we conduct experiments on GR00T in Appendix Sec. H, which validates that our algorithm can be applied to other flow-based VLAs.

5.2. Main Results

In this section, we assess the in-distribution (ID) performance of π_{RL} across various benchmarks, followed by an analysis of its out-of-distribution (OOD) generalization.

5.2.1. IN-DISTRIBUTION RL TRAINING

As detailed in Tab. 1, π_{RL} yields substantial performance gains over SFT baselines across all evaluated benchmarks. Specifically, the π_0 model achieves a maximum average improvement of +29.2%, while the $\pi_{0.5}$ variant demonstrates a +31.0% increase in average success rate.

Specifically for LIBERO, we perform few-shot SFT on the $\pi_{0.5}$ model followed by RL optimization to achieve a 98.3% success rate, outperforming the 96.9% success rate of the full-dataset SFT baseline. These performance gains extend to other challenging environments, including ManiSkill with

Table 1. Comprehensive ID performance comparison across four benchmarks.

Model		Benchmarks					
		LIBERO	ManiSkill	MetaWorld	CALVIN	Avg.	Δ Avg.
π_0	SFT	57.6	38.4	50.8	57.5	51.1	—
	Flow-SDE	96.1	78.8	78.1	61.7	78.7	+27.6
	Flow-Noise	97.6	77.8	85.8	59.9	80.3	+29.2
$\pi_{0.5}$	SFT	77.1	40.1	43.8	61.3	55.6	—
	Flow-SDE	97.9	90.9	70.7	87.0	86.6	+31.0
	Flow-Noise	98.3	89.7	66.1	84.5	84.7	+29.1

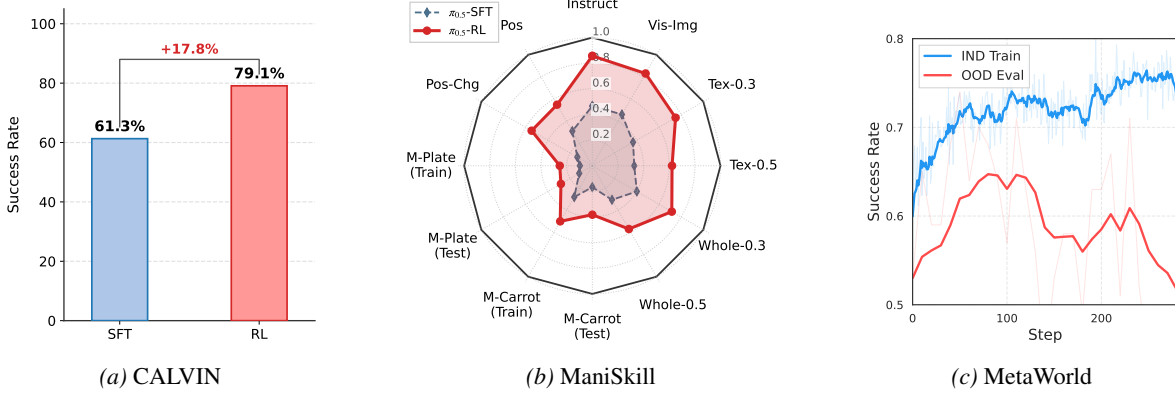


Figure 5. Comprehensive OOD evaluation results on CALVIN ABC-D, ManiSkill OOD, and MetaWorld ML45 benchmarks.

its 4,352 pick-and-place task combinations, MetaWorld featuring 50 distinct manipulation primitives, and CALVIN for long-horizon sequential tasks. See Appendix Sec. C for comprehensive experimental details.

5.2.2. OUT-OF-DISTRIBUTION RL EVALUATION

While previous experiments demonstrate that RL yields performance improvements, a critical question remains: *does RL yield an enhanced policy, or simply overfit to the ID environment driven by provided rewards?* In this section, we evaluate RL-finetuned policies in OOD scenarios, where the environment distribution or the task objective deviates from the ID training setup, to assess their generalization capabilities.

As illustrated in Fig. 5, the performance gains achieved in the ID setting effectively transfer to OOD scenarios in ManiSkill and CALVIN, where the domain shift primarily stems from environmental variations. Conversely, for the OOD setting in MetaWorld, which involves distinct manipulation tasks, performance fluctuates without showing significant improvement. This finding suggests that the benefits of RL are primarily localized to action-level refinement rather than broader augmentation of cross-task generalization capabilities. See Appendix Sec. D for more details.

5.3. Ablation Study

Given that Flow-SDE achieves performance comparable to Flow-Noise while offering higher computational efficiency, we conduct our ablation studies with the Flow-SDE method. Specifically, we investigate the impact of critic designs, noise injection strategies, and MDP formulations, with additional results on RL algorithms and hyper-parameters provided in Appendix Sec. F.

5.3.1. CRITIC DESIGN

Placement. We compare two critic placement strategies, one positioned after the action expert (V_{expert}) and the other after the VLM (V_{vlm}), with π_0 model on the LIBERO-Long task suite. As illustrated in Fig. 6, we observe that V_{vlm} exhibits slightly superior performance, lower value loss, and higher explained variance, despite not receiving the proprioceptive state as input. This advantage can be attributed to a key difference in their input: V_{vlm} learns a direct mapping from observation to value, while V_{expert} must contend with optimization challenges arising from coupled state and noisy action inputs.

Nevertheless, to align with the concept of the value function, we maintain the V_{expert} architecture for the π_0 , ensuring that state information is incorporated to estimate the value.

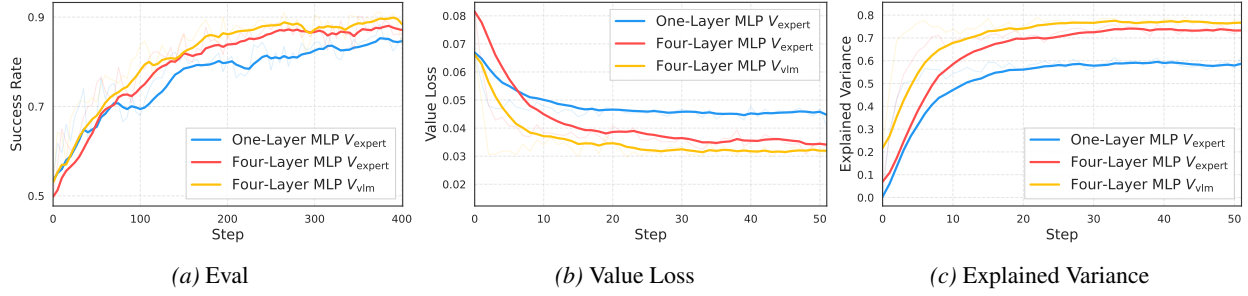


Figure 6. Ablation on the critic design within Flow-SDE π_0 on the LIBERO-Long, indicating that the critic V_{vlm} attached after the VLM exhibits superior performance. Furthermore, a four-layer MLP demonstrates stronger regression capability in V_{expert} .

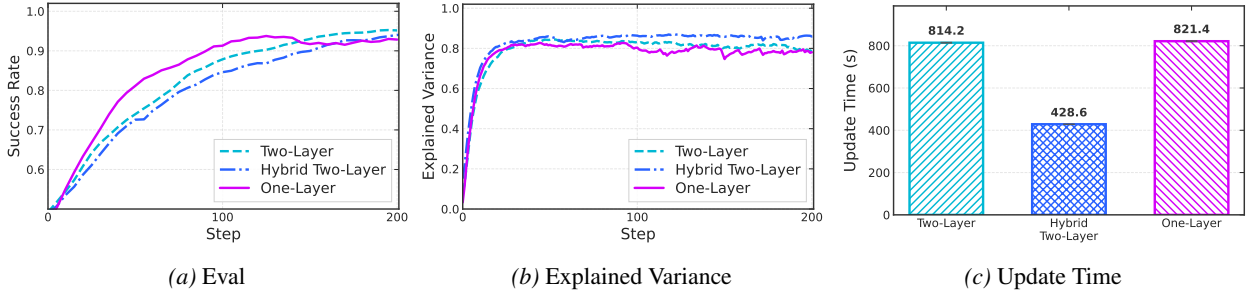


Figure 7. Ablation on the MDP formulation within Flow-SDE of π_0 on the LIBERO-Goal.

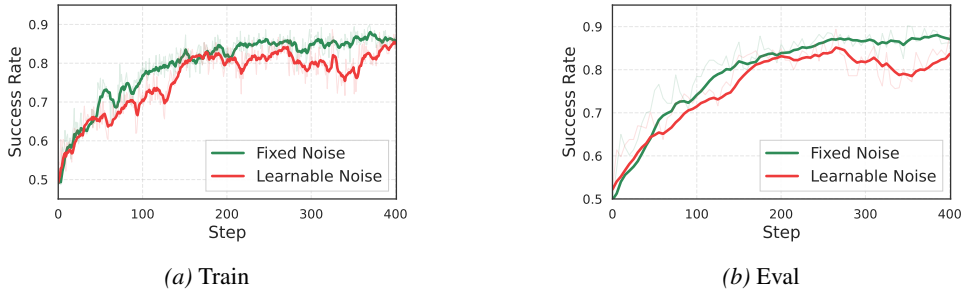


Figure 8. Ablation on the injection strategy within Flow-SDE of π_0 on the LIBERO-Long.

Structure. We investigate a four-layer MLP versus a one-layer MLP, which mirrors the action-projection structure in the action expert. Results in Fig. 6 indicate that the four-layer MLP leads to a more accurate value approximation, resulting in enhanced performance and training stability.

5.3.2. FLOW POLICY MDP

With the same fixed noise injection strategy, we evaluate the one-layer MDP of Flow-Noise with the two-layer MDP of Flow-SDE on the LIBERO-Goal, as shown in Fig. 7.

We observe that the one-layer formulation converges most rapidly, but the final success rates remain consistent across all three formulations. In terms of computational efficiency, the hybrid two-layer paradigm achieves a $2\times$ speedup over the standard approach due to its shorter effective MDP chain. Notably, the one-layer MDP yields no substantial wall-clock

time advantage over the standard two-layer model, stemming from the requirement to recalculate full denoising trajectories for log likelihood estimation.

5.3.3. STOCHASTICITY INJECTION

We compare fixed and learnable noise injection strategies using the Flow-SDE MDP formulation on the LIBERO-Long suite. To ensure a controlled comparison, we set the entropy bonus for the learnable noise to zero, aligning it with the fixed noise approach.

Specifically, we set the fixed noise to 0.5, and lower and upper bounds for the learnable noise log-variance to 0.08 and 0.16. As depicted in Fig. 8, two noise strategies exhibit similar train performance at step 0, which indicates comparable noise magnitudes. Furthermore, the converged performance affirms the efficiency of both injection methods.

Table 2. Ablation study of hyperparameters for Flow-SDE on the LIBERO-Spatial. *Train* refers to policy performance during the stochastic rollout phase, whereas *Eval* refers to performance during the deterministic evaluation phase.

Models	Stage	Hyperparameters									
		Noise Level			Denoise Step				Action Chunk		
		0.2	0.5	0.8	1	2	4	8	5	10	20
SFT	Train	62.3	56.0	46.6	9.4	28.3	56.1	62.6	56.0	60.7	70.3
	Eval	65.2	65.2	65.2	63.8	64.9	65.2	63.2	65.2	70.5	72.6
RL	Train	59.5	93.5	95.3	73.8	90.8	93.5	84.3	93.5	93.3	87.5
	Eval	73.1	94.5	98.1	88.5	97.0	94.5	86.7	94.5	95.5	89.2

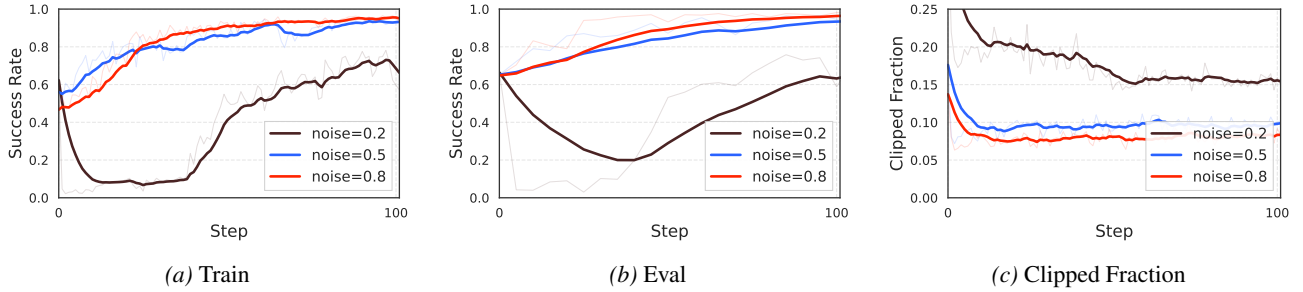


Figure 9. Ablation on the noise level a , conducted with the Flow-SDE π_0 on the LIBERO-Spatial.

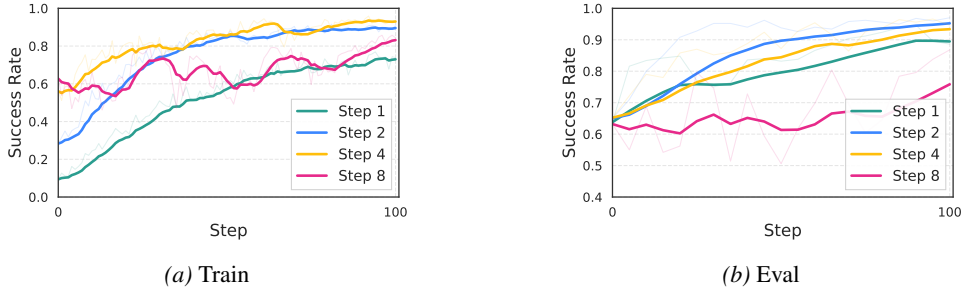


Figure 10. Ablation on the denoise step, conducted with the Flow-SDE π_0 on the LIBERO-Spatial.

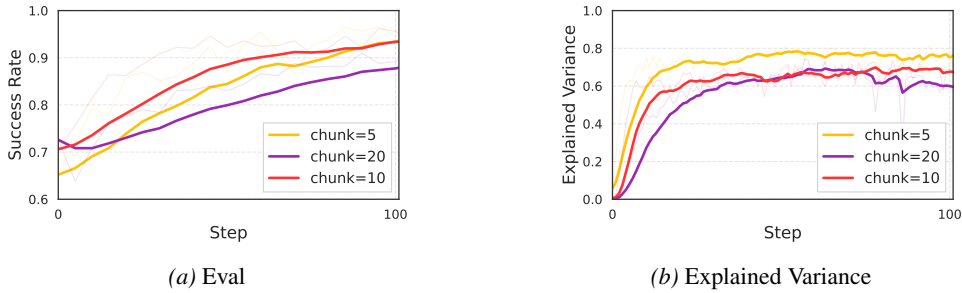


Figure 11. Ablation on the chunk size, conducted with the Flow-SDE π_0 on the LIBERO-Spatial.

5.4. Hyper-Parameters

Building on the Flow-SDE with π_0 model, we investigate the influence of the noise level, denoise step, and action chunk on the LIBERO-Spatial benchmark. We denote the train stage as the phase where the policy generates stochastic

actions for exploration, whereas the evaluation stage involves generating deterministic actions. The train and eval success rates for the SFT baseline and the RL fine-tuned model after 100 training steps are presented in Tab. 2.

Noise Level. The noise level a in the Flow-SDE is defined in

Eq. (8), which governs the noise injection magnitude during the denoising process. As shown in Tab. 2, the evaluation performance of the SFT baseline remains identical across all noise levels due to its reliance on deterministic ODE sampling. Conversely, its training performance exhibits a clear degradation as noise increases. This is intuitive, as higher noise levels can distort the flow trajectory, leading to an inaccurate estimation of the marginal action distribution.

Extending this analysis to the RL fine-tuning stage highlights a critical trade-off: *while lower noise levels mitigate exploration-induced performance degradation, they simultaneously constrain the capacity for RL refinement*. This trade-off is empirically supported by Fig. 9, which shows that training with minimal noise ($\alpha = 0.2$) leads to instability, characterized by a significantly higher clip fraction. We attribute this instability to the substantially larger gradient magnitudes associated with low-noise regimes.

Denoise Step. The denoise step K defines the number of discretization steps for action generation and is critical for controlling the fidelity of the ODE-to-SDE transition in Eq. (8). In Tab. 2, we observe that while all configurations start with similar eval performance, the train success rate plummets at $K = 1$, indicating a significant ODE-to-SDE discretization error.

However, a larger K is not unequivocally optimal. As illustrated in Fig. 10, increasing K presents a distinct trade-off: *while it enhances rollout performance, it simultaneously increases training complexity and computational overhead due to the extended sequence of denoising steps*.

Action chunk. The action chunk refers to the number of consecutive actions the policy executes within a single observation. We ablate the action chunk size across 5, 10, and 20, with results visualized in Fig. 11.

Although a larger chunk size yields marginal performance gains, it inherently reduces the frequency of policy-environment interactions and obscures precise reward credit assignment. These constraints lead to less reliable advantage estimation, as evidenced by the diminished explained variance. Consequently, while an increased chunk size may offer a superior SFT baseline, it paradoxically limits the ceiling for subsequent RL-driven refinement.

6. Conclusion

We introduce π_{RL} , a framework that enables flow-based VLAs, π_0 and $\pi_{0.5}$, to be fine-tuned with online RL algorithms. We tackle the fundamental challenge of intractable log-likelihoods in flow matching with Flow-Noise and Flow-SDE solutions. Our extensive experiments on the challenging benchmarks demonstrated that π_{RL} achieves significant performance improvements over SFT baselines.

Limitation. Due to the low sample efficiency of online RL, our framework currently relies on sim-to-real deployment. We aim to develop more efficient algorithms to enable real-world RL training in the future.

References

- Beyer, L., Steiner, A., Pinto, A. S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschanen, M., Bugliarello, E., et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Chen, G., Li, Z., Wang, S., Jiang, J., Liu, Y., Lu, L., Huang, D.-A., Byeon, W., Le, M., Rintamaki, T., et al. Eagle 2.5: Boosting long-context post-training for frontier vision-language models. *arXiv preprint arXiv:2504.15271*, 2025.
- Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11): 1684–1704, 2025.
- Din, M. U., Akram, W., Saoud, L. S., Rosell, J., and Hussain, I. Vision language action models in robotic manipulation: A systematic review. *arXiv preprint arXiv:2507.10672*, 2025.
- Fan, H., Dai, H., Zhang, J., Li, J., Yan, Q., Zhao, Y., Gao, M., Wu, J., Tang, H., and Dong, H. Twinaligner: Visual-dynamic alignment empowers physics-aware real2sim2real for robotic manipulation. *arXiv preprint arXiv:2512.19390*, 2025.
- Fei, S., Wang, S., Shi, J., Dai, Z., Cai, J., Qian, P., Ji, L., He, X., Zhang, S., Fei, Z., et al. Libero-plus: In-depth robust-

- ness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- Firoozi, R., Tucker, J., Tian, S., Majumdar, A., Sun, J., Liu, W., Zhu, Y., Song, S., Kapoor, A., Hausman, K., et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, 44(5):701–739, 2025.
- Guo, R., Lin, X., Liu, M., Gu, J., and Su, H. Mplib: a lightweight motion planning library. <https://github.com/haosulab/MPlib>, 2025. URL <https://motion-planning-lib.readthedocs.io/latest/>.
- He, X., Fu, S., Zhao, Y., Li, W., Yang, J., Yin, D., Rao, F., and Zhang, B. Tempflow-grpo: When timing matters for grpo in flow models. *arXiv preprint arXiv:2508.04324*, 2025.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Hutchinson, M. F. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Jiang, G., Chang, H., Qiu, R.-Z., Liang, Y., Ji, M., Zhu, J., Dong, Z., Zou, X., and Wang, X. Gsworld: Closed-loop photo-realistic simulation suite for robotic manipulation. *arXiv preprint arXiv:2510.20813*, 2025.
- Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sankeketi, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Li, H., Zuo, Y., Yu, J., Zhang, Y., Yang, Z., Zhang, K., Zhu, X., Zhang, Y., Chen, T., Cui, G., et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025a.
- Li, J., Cui, Y., Huang, T., Ma, Y., Fan, C., Yang, M., and Zhong, Z. Mixgrpo: Unlocking flow-based grpo efficiency with mixed ode-sde. *arXiv preprint arXiv:2507.21802*, 2025b.
- Li, X., Hsu, K., Gu, J., Pertsch, K., Mees, O., Walke, H. R., Fu, C., Lunawat, I., Sieh, I., Kirmani, S., et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- Li, Y., Wang, Y., Zhu, Y., Zhao, Z., Lu, M., She, Q., and Zhang, S. Branchgrpo: Stable and efficient grpo with structured branching in diffusion models. *arXiv preprint arXiv:2509.06040*, 2025c.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Liu, J., Gao, F., Wei, B., Chen, X., Liao, Q., Wu, Y., Yu, C., and Wang, Y. What can rl bring to vla generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025a.
- Liu, J., Liu, G., Liang, J., Li, Y., Liu, J., Wang, X., Wan, P., Zhang, D., and Ouyang, W. Flow-grpo: Training flow matching models via online rl. *arXiv preprint arXiv:2505.05470*, 2025b.
- Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
- McAllister, D., Ge, S., Yi, B., Kim, C. M., Weber, E., Choi, H., Feng, H., and Kanazawa, A. Flow matching policy gradients. *arXiv preprint arXiv:2507.21053*, 2025.
- McLean, R., Chatzaroulas, E., McCutcheon, L., Röder, F., Yu, T., He, Z., Zentner, K., Julian, R., Terry, J., Woungang, I., et al. Meta-world+: An improved, standardized, rl benchmark. *arXiv preprint arXiv:2505.11289*, 2025.
- Mees, O., Hermann, L., Rosete-Beas, E., and Burgard, W. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

- O'Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlikar, A., Jain, A., et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903. IEEE, 2024.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4195–4205, 2023.
- Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., and Levine, S. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741, 2023.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- Tan, S., Dou, K., Zhao, Y., and Krähenbühl, P. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025.
- Tao, S., Xiang, F., Shukla, A., Qin, Y., Hinrichsen, X., Yuan, X., Bao, C., Lin, X., Liu, Y., Chan, T.-k., et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.
- Team, O. M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Wang, F. and Yu, Z. Coefficients-preserving sampling for reinforcement learning with flow matching. *arXiv preprint arXiv:2509.05952*, 2025.
- Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- Yu, C., Wang, Y., Guo, Z., Lin, H., Xu, S., Zang, H., Zhang, Q., Wu, Y., Zhu, C., Hu, J., et al. Rlinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025.
- Zang, H., Wei, M., Xu, S., Wu, Y., Guo, Z., Wang, Y., Lin, H., Shi, L., Xie, Y., Xu, Z., et al. Rlinf-vla: A unified and efficient framework for vla+ rl training. *arXiv preprint arXiv:2510.06710*, 2025.
- Zhang, T., Yu, C., Su, S., and Wang, Y. Reinflow: Fine-tuning flow matching policy with online reinforcement learning. *arXiv preprint arXiv:2505.22094*, 2025.

A. Appendix

This appendix provides additional technical details and experimental results for π_{RL} . The content is organized as:

- **Appendix B:** Experimental Setup.
- **Appendix C:** In-Distribution RL Training.
- **Appendix D:** Out-of-Distribution RL Evaluation.
- **Appendix E:** Case Studies: Single-Task RL Training.
- **Appendix F:** Ablation Details.
- **Appendix G:** Insights from Large-Scale RL Training.
- **Appendix H:** RL for GR00T N1.5.
- **Appendix I:** Limitations and Future Work.
- **Appendix J:** Training Hyperparameters.

B. Experimental Setup

B.1. Benchmarks.

To rigorously assess the performance and generalization of our framework, we conduct evaluations across four diverse benchmarks, with diverse emphases on robotic capability:

- **LIBERO** focuses on compositional variations and knowledge transfer in tasks. By evaluating across four task suites: *Spatial*, *Object*, *Goal*, and *Long*, it probes the model’s ability to transfer base skills to variations in object arrangements and long-horizon tasks.
- **ManiSkill** emphasizes perceptual and execution robustness under massive environmental diversity. We follow the setup of RL4VLA (Liu et al., 2025a) and built a benchmark with 4,352 unique pick-and-place combinations, which challenges the policy to maintain physical interactions across a vast distribution of objects and receptacles.
- **CALVIN** evaluates long-horizon sequential reasoning and vision-language grounding. It evaluates the model’s capacity to execute chains of five random sub-tasks in a persistent environment, requiring accurate alignment with linguistic instructions.
- **MetaWorld** measures skill breadth and multi-task versatility. It requires a single policy to master 50 semantically distinct manipulation primitives, ranging from simple reaching to complex tool usage.

B.2. Implementation Details.

Given that pre-trained models often struggle to generalize to task-specific benchmarks, we initiate our process with SFT on expert demonstrations. For the SFT stage, we fine-tune the entire 3.3B model following the official setting. In the subsequent RL stage, we freeze the VLM parameters and exclusively fine-tune the 300M action expert model, driven by GPU memory efficiency and the findings from RL4VLA that RL contributes more significantly to action generalization. We build the whole framework upon the RLinf (Yu et al., 2025) codebase, where we adopt a shared, co-located GPU allocation strategy that places the environment, rollout model, and actor model on the same GPU and executes them serially.

For the model configurations, we adhere to the official setting provided by openpi (Black et al., 2024; Intelligence et al., 2025). In these settings, π_0 utilizes image, language, and proprioceptive states as input, whereas $\pi_{0.5}$ notably omits state information². Our experiments are conducted on 8 NVIDIA H100 80GB GPUs, and detailed training hyperparameters are available in the Appendix Sec. J.

C. In-Distribution RL Training

In this section, we evaluate the effectiveness of our framework across various benchmarks to demonstrate its robustness and superior performance in ID settings.

C.1. LIBERO Benchmark

SFT Procedure. The LIBERO benchmark comprises four task suites, each consisting of 10 distinct subtasks. To facilitate few-shot SFT on LIBERO, a minimum of 40 expert demonstration trajectories is necessary to ensure a positive success rate for each subtask across four task suites, thereby guaranteeing a positive optimization signal for the subsequent RL phase.

We perform few-shot SFT following the official training configurations. For the π_0 model, we fine-tune on a subset of 58 trajectories sampled from the 1,692 total demonstrations³, which serves as the initial checkpoint for subsequent RL training on the *Spatial*, *Object*, and *Goal* task suites. For the *Long* task suite, a larger pool of 208 trajectories is employed to address its more challenging, long-horizon nature. In contrast, for the $\pi_{0.5}$ model, benefiting from a superior pre-trained checkpoint and training configurations, we leverage only 40 trajectories to provide a unified few-shot SFT checkpoint across all task suites.

RL Procedure. In RL, the VLA model receives a multi-modal input state comprising: an agent-view and a wrist-

²<https://github.com/Physical-Intelligence/openpi/issues/687>

³<https://huggingface.co/datasets/physical-intelligence/libero>

Table 3. Evaluation results on the LIBERO benchmark, evaluated based on the success rate (%).

Model	LIBERO					
	Spatial	Object	Goal	Long	Avg.	Δ Avg.
# Full Dataset SFT						
Octo	78.9	85.7	84.6	51.1	75.1	—
OpenVLA	84.7	88.4	79.2	53.7	76.5	—
π_{fast}	96.4	96.8	88.6	60.2	85.5	—
OpenVLA-OFT	91.6	95.3	90.6	86.5	91.0	—
π_0	96.8	98.8	95.8	85.2	94.2	—
$\pi_{0.5}$	98.8	98.2	98.0	92.4	96.9	—
# Few-shot SFT + RL						
π_0	SFT	65.3	64.4	49.8	57.6	—
	Flow-SDE	98.4	99.4	96.2	90.2	+38.5
	Flow-Noise	99.0	99.2	98.2	93.8	+40.0
# Few-shot SFT + RL						
$\pi_{0.5}$	SFT	84.6	95.4	84.6	43.9	—
	Flow-SDE	99.6	100	98.8	93.0	+20.8
	Flow-Noise	99.6	100	99.6	94.0	+21.2

Table 4. Evaluation results on the ManiSkill benchmark, with more specific OOD results depicted in Tab. 7.

Model		IND		OOD				
		Avg.	Δ	Vision	Semantic	Execution	Avg.	Δ
π_0	SFT	38.4	—	32.6	8.4	13.2	18.1	—
	Flow-SDE	78.8	+40.4	61.1	25.4	31.5	39.3	+21.2
	Flow-Noise	77.8	+39.4	63.4	23.1	24.2	36.9	+18.8
$\pi_{0.5}$	SFT	40.1	—	40.2	16.6	22.4	26.4	—
	Flow-SDE	90.9	+50.8	68.0	34.5	45.4	49.3	+22.9
	Flow-Noise	89.7	+49.6	69.9	35.5	54.9	53.4	+27.0

view, natural language guidance, the robot end effector pose, and the gripper state. The model outputs an action to interact with the LIBERO environment, which provides a binary reward of 1 for successful task completion and 0 otherwise.

Experiments. We benchmark the performance of π_{RL} , which fine-tunes the few-shot SFT π_0 and $\pi_{0.5}$ models with Flow-Noise and Flow-SDE, against several state-of-the-art VLAs trained on the entire LIBERO dataset, including Octo, OpenVLA, OpenVLA-OFT, π_{fast} (Pertsch et al., 2025), π_0 , and $\pi_{0.5}$. We conduct experiments on four LIBERO task suites and report performance as the success rate across all 500 initial states (10 sub-tasks $\times$ 50 states each).

Analysis. For the few-shot π_0 model, the SFT baseline performs poorly, with an average success rate of only 57.6%, indicating that the model struggles with limited demonstration data. π_{RL} substantially boosts performance, with Flow-SDE and Flow-Noise reaching 96.1% and 97.6%, and surpassing the full-dataset π_0 SFT baseline of 94.2%.

While the $\pi_{0.5}$ few-shot SFT baseline achieves a decent average performance of 77.1%, it struggles with the challenging *Long* task suite, scoring only 43.9%. Our proposed π_{RL} rectifies this deficiency, boosting the *Long* task success rate from 43.9% to 94.0%, constituting a 50.1% improvement. Notably, despite using only a single trajectory for SFT, π_{RL} reaches 98.3% final performance, surpassing the 96.9% full-dataset SFT model.

C.2. ManiSkill Benchmark

SFT Procedure. In the ManiSkill benchmark, the policy is required to pick from 16 object types and place them onto 17 receptacles across 16 unique table scenes, yielding 4,352 unique task combinations. Given the high complexity of this setting, the SFT dataset consists of 16,384 episodes synthesized using the MPLib motion planning suite (Guo et al., 2025). To reinforce the concept of motion completion, 15 additional frames are appended to the end of each trajectory.

Table 5. Evaluation results on the CALVIN benchmark (Scene D), reporting the average completed subtasks and success rates for task sequences of length 1 to 5.

Methods		CALVIN-D						
		Len-1	Len-2	Len-3	Len-4	Len-5	Avg.	Δ Avg.
π_0	SFT	94.7	84.9	74.3	65.2	57.5	3.766	—
	Flow-SDE	96.4	88.0	77.5	70.8	61.7	3.944	+0.178
	Flow-Noise	96.9	88.8	78.0	68.3	59.9	3.919	+0.153
$\pi_{0.5}$	SFT	92.7	84.3	76.7	68.8	61.3	3.838	—
	Flow-SDE	99.7	98.2	95.8	91.0	87.0	4.717	+0.879
	Flow-Noise	99.6	97.6	93.9	89.6	84.5	4.652	+0.814

Table 6. Evaluation results on the MetaWorld MT50 benchmark.

Methods		MetaWorld-MT50					
		Easy	Medium	Hard	Very Hard	Avg.	Δ Avg.
Diffusion Policy		23.1	10.7	1.9	6.1	10.5	—
TinyVLA		77.6	21.5	11.4	15.8	31.6	—
SmolVLA		87.1	51.8	70.0	64.0	68.2	—
π_0	SFT	77.9	51.8	53.3	20.0	50.8	—
	Flow-SDE	92.1	74.6	61.7	84.0	78.1	+27.3
	Flow-Noise	91.1	81.8	78.3	92.0	85.8	+35.0
$\pi_{0.5}$	SFT	68.2	37.3	41.7	28.0	43.8	—
	Flow-SDE	86.4	55.5	75.0	66.0	70.7	+26.9
	Flow-Noise	86.8	58.1	63.3	56.0	66.1	+22.3

RL Procedure. In RL, the VLA model receives a third-person RGB image, a concise language instruction, and the current joint proprioception. The environment provides a structured reward signal: 1.0 for correct object placement and an auxiliary 0.1 reward for successful gripper-object attachment, intended to encourage stable manipulation and mitigate undesired behaviors such as impulsive throwing.

Experiments. Following the RL4VLA experimental protocol, we conduct RL training on a comprehensive set of 4,352 task combinations and record the performance as the aggregate success rate across these tasks.

Analysis. As detailed in Tab. 4, π_{RL} significantly boosts performance in the training environment. Specifically, the success rate of π_0 increases from 38.4% to 77.8%, while $\pi_{0.5}$ improves from 40.1% to 90.9%. These gains underscore the efficacy of RL in complex settings.

C.3. CALVIN Benchmark

SFT Procedure. We conduct SFT on the CALVIN ABC dataset⁴, which comprises approximately 24 hours of un-

⁴https://huggingface.co/datasets/InternRobotics/InternData-Calvin_ABC

structured "play data" across three distinct environments (A, B, and C). This dataset includes over 20,000 language-labeled trajectories covering 34 unique manipulation tasks.

RL Procedure. Each episode consists of a sequence of five randomly sampled subtasks to be completed in succession without environment resets. The reward signal is defined at the subtask level, where the model receives a sparse binary reward of 1.0 for each successfully executed subtask and 0.0 otherwise.

Experiments. We evaluate the performance of π_{RL} in Scene D over 1,000 episodes. Following the standard CALVIN evaluation protocol, we report two key metrics: (1) the success rate for task sequences of increasing lengths, namely *Len-1* to *Len-5*. (2) the average number of completed subtasks, denoted as *Avg.* per episode.

Analysis. As detailed in Tab. 5, π_{RL} yields substantial performance gains in long-horizon sequential execution, particularly with the $\pi_{0.5}$ variant. The SFT models inherently struggle with compounding errors across sequential tasks, with $\pi_{0.5}$ only achieving a 61.3% success rate on *Len-5* sequences. RL effectively mitigates this issue with the average completed sub-tasks of Flow-SDE increasing from 3.838 to 4.717, and its *Len-5* success rate surges to 87.0%.

Table 7. Specific generalization evaluation results in the ManiSkill OOD setting.

Environment	Variation-Version-Type	π_0 -SFT	π_0 -RL Flow-SDE	π_0 -RL Flow-Noise	$\pi_{0.5}$ -SFT	$\pi_{0.5}$ -RL Flow-SDE	$\pi_{0.5}$ -RL Flow-Noise
In distribution	Main-v3-train	38.4	78.8	77.8	40.1	90.9	89.7
Visual-Language Variations	Instruct-v1-test	30.1	64.6	66.5	46.6	77.0	85.7
	VisionImage-v1-test	38.3	68.8	71.7	46.2	78.8	83.1
	VisionTexture03-v1-test	35.1	66.0	66.8	36.7	69.6	75.0
	VisionTexture05-v1-test	31.0	55.8	60.5	32.7	58.0	62.2
	VisionWhole03-v1-test	35.4	62.4	69.0	40.1	69.6	71.6
	VisionWhole05-v1-test	28.5	49.0	53.9	30.7	55.0	57.0
Semantic Reasoning (object/receptacle confounders)	MultiCarrot-v1-test	7.8	28.2	23.0	16.7	36.8	38.2
	MultiCarrot-v1-train	12.5	36.5	31.8	28.2	49.5	50.1
	MultiPlate-v1-test	5.0	16.4	18.3	11.8	29.4	28.3
	MultiPlate-v1-train	7.3	20.5	19.6	9.7	22.3	25.4
Action Execution	PositionChangeTo-v1-test	9.6	17.4	10.9	13.5	36.2	54.7
	Position-v1-test	16.9	45.6	37.5	31.2	54.5	55.0

Notably, the performance gap between SFT and RL widens significantly as the sequence length increases. For $\pi_{0.5}$, while Flow-SDE shows a modest 7.0% improvement over SFT in Len-1 tasks, the gap expands to an impressive 25.7% in the most challenging Len-5 sequences.

C.4. MetaWorld Benchmark

SFT Procedure. We perform SFT on the π_0 and $\pi_{0.5}$ models using the official dataset⁵, which consists of 2500 trajectories across 50 different manipulation tasks.

RL Procedure. During the RL procedure, the VLA model processes a multi-modal input comprising a RGB agent-view image, language guidance, the robot’s end-effector position, and its gripper state. Based on this input, the model outputs an action to interact with the environment, which in turn provides a sparse reward: 1 for successful task completion and 0 otherwise.

Experiments. We benchmark the performance of π_{RL} against Diffusion Policy (Chi et al., 2025), TinyVLA (Wen et al., 2025), and SmolVLA (Shukor et al., 2025). For the performance evaluation, we follow the setup from SmolVLA, i.e., classifying 50 tasks into *easy*, *medium*, *hard*, and *very hard* four categories according to their difficulties.

Analysis. As detailed in Tab. 6, RL fine-tuning substantially boosts performance. The π_0 and $\pi_{0.5}$ models achieve average success rates of 85.8% and 70.7%, respectively. This marks a significant improvement over their SFT-only counterparts and surpasses the SmolVLA baseline of 68.2%, confirming that RL can effectively enhance model capabilities across a diverse range of manipulation task types.

⁵https://huggingface.co/datasets/lerobot/metaworld_mt50

D. Out-of-Distribution RL Evaluation

While previous experiments demonstrate significant RL-driven improvements in the ID domain, this section evaluates OOD generalization. As LIBERO lacks a dedicated interface for OOD testing, we utilize the ManiSkill, CALVIN, and MetaWorld benchmarks to investigate *whether the RL-driven improvements represent genuine skill acquisition that scales to novel settings, or merely reflect the exploitation of environment-specific biases*.

D.1. ManiSkill

Setup. Following RL4VLA, we evaluate the model’s generalization across three challenging OOD scenarios: (1) *Vision*, challenging the model with novel backgrounds and textures; (2) *Semantics*, probing comprehension with unseen objects, varied instructions, and confounding elements like extra objects or receptacles; (3) *Execution*, assessing robustness against varied initial states, unseen robot poses, and dynamic disturbances.

Results. In the OOD scenarios detailed in Tabs. 4 and 7, we observe that the π_0 -SFT model demonstrates strong generalization for visual information. This can be attributed to the robust foundation of its VLM, which allows it to handle visual disturbances better.

However, the semantic performance of π_0 drops dramatically. This degradation is less pronounced when switching to the $\pi_{0.5}$ baseline, a benefit likely stemming from the knowledge generalization of the pre-trained $\pi_{0.5}$ model. Regarding action execution, π_0 exhibits a larger performance drop than $\pi_{0.5}$. We hypothesize that this discrepancy arises from the inclusion of joint angle states as input in π_0 , leading to severe overfitting in the control task. In contrast, $\pi_{0.5}$

Table 8. Evaluation results on the SIMPLER benchmark for π_0 and $\pi_{0.5}$ with Flow-Noise method.

Model		SIMPLER				
		Carrot	Eggplant	Spoon	Cube	Avg.
π_0	SFT	82.7	87.5	61.7	37.1	67.2
	+RL	95.7	96.7	91.6	63.0	86.7
	Δ	+13.0	+9.2	+29.9	+25.9	+19.5
$\pi_{0.5}$	SFT	70.6	91.9	43.5	31.0	59.2
	+RL	82.0	98.2	82.8	53.3	79.1
	Δ	+11.4	+6.3	+39.3	+22.3	+19.9



Figure 12. Real-world deployment of an RL refined policy performing a pick and place task.

omits these inputs, thereby avoiding the same degree of performance degradation.

As for the RL training, although the performance improvements in OOD scenarios are lower than those in IND settings, the proportional improvements achieved are notably comparable. As indicated in Tab. 4, for the $\pi_{0.5}$ model, Flow-SDE enhances the IND success rate by 126.7%, while the OOD similarly increases by 102.3%. This consistency in relative gains indicates that RL-driven optimization promotes the acquisition of generalized action representations rather than merely overfitting the training environment, thus preserving efficacy under distribution shifts.

Nevertheless, we observe a performance gap in relative improvement between the *Vision* OOD tasks and the IND domain. Specifically, for $\pi_{0.5}$, the 73.9% gain in visual generalization trails the 126.7% increase observed in the training environment. This discrepancy likely stems from freezing the VLM backbone during the RL stage for computational efficiency, which restricts the model’s ability to adapt its visual grounding features to novel textures and backgrounds.

D.2. CALVIN

Setup. We evaluate environmental and visual OOD robustness based on the $\text{ABC} \rightarrow \text{D}$ protocol in CALVIN. Under this setting, the model is trained on ABC environments and evaluated in a zero-shot manner on Scene D. Scene D introduces significant distribution shifts in terms of visual textures, lighting conditions, and spatial layouts, effectively assessing the agent’s ability to transfer skills to an unfamiliar physical environment.

Results. Under identical $\text{D} \rightarrow \text{D}$ training settings, the RL finetuned policy in the ABC environment reaches a 79.1% success rate in the OOD scene D, improving over the 61.3% SFT baseline as shown in Fig. 5. Aligned with the findings in ManiSkill, this suggests that ID gains can be transferred to OOD settings characterized by visual variations.

D.3. MetaWorld

Setup. We utilize the ML45 benchmark from MetaWorld to evaluate the task-level generalization. This setup consists of 50 distinct robotic manipulation tasks: the agent is trained on 45 base tasks and subsequently evaluated on 5 held-out, unseen tasks, which require the model to generalize its learned manipulation primitives to entirely novel task objectives and workspace configurations.

Results. As evidenced in the Fig. 5, while success rates show consistent gains within the ID domain, OOD performance is characterized by persistent oscillation throughout the training process. This instability indicates that RL, in its current form, struggles to foster stable cross-category generalization.

Nevertheless, the model retains the OOD skills learned during the SFT phase throughout the RL training process. This highlights a significant advantage over standard SFT, which often causes the model to overfit on expert demonstrations and lose its broader capabilities (Li et al., 2025a). Unlike standard SFT, RL enables ID performance gains while preserving the general knowledge, which indicates that RL provides a more balanced optimization that maintains OOD robustness without catastrophic forgetting.

Table 9. Comparison of the PPO and GRPO with Flow-SDE on the LIBERO.

Model		LIBERO					
		Spatial	Object	Goal	Long	Avg.	Δ Avg.
π_0	SFT	65.3	64.4	49.8	51.2	57.6	—
	+GRPO	97.8	97.8	83.2	81.4	90.0	+32.4
	+PPO	98.4	99.4	96.2	90.2	96.0	+38.4
$\pi_{0.5}$	SFT	84.6	95.4	84.6	43.9	77.1	—
	+GRPO	97.4	99.8	91.2	77.6	91.5	+14.4
	+PPO	99.6	100	98.8	93.0	97.9	+20.8

D.4. Summary

In conclusion, our OOD evaluation demonstrates that *RL enhances performance for similar tasks but fails to generalize effectively to novel task objectives*.

Specifically, RL training effectively enhances robustness against low-level variations such as the visual and execution shifts observed in ManiSkill and CALVIN. This indicates that the model acquires generalized action representations rather than merely overfitting to the training environment.

Regarding high-level generalization on MetaWorld, the model successfully retains the OOD skills inherited from the SFT phase, demonstrating that RL avoids the catastrophic forgetting and overfitting typical of standard imitation learning. However, transferring its performance gains to entirely novel task objectives remains a significant challenge.

E. Case Studies: Single-Task RL Training

While the preceding experiments focused on performance across multi-task benchmarks, this section investigates single-task scenarios where the VLA is trained to master a specific task within a relatively static environment. Specifically, we evaluate our approach on the **SIMPLER** (Li et al., 2024) and a **Real2Sim2Real** environment.

E.1. SIMPLER

Setup. In SIMPLER, the experimental setup comprises an 8-DoF WidowX-250S arm evaluated on four standard tasks: (1) Spoon: placing a spoon on a cloth. (2) Carrot: placing a carrot on a plate. (3) Eggplant: placing an eggplant in a basket. (4) Cube: stacking a cube. For the SFT stage, we employ a curated dataset in which each task is trained with 144 demonstration episodes.

Analysis. As detailed in Tab. 8, π_{RL} increases the average success rate of the π_0 model from 67.2% to 86.7%, with three tasks (carrot, eggplant, and spoon) exceeding 90% success.

E.2. Real2Sim2Real

While the SIMPLER benchmark demonstrates predictive correlation between simulation and real-world performance, a pronounced visual domain shift remains to be solved. To this end, we leverage recent Real2Sim2Real methodologies (Fan et al., 2025; Jiang et al., 2025) to construct a high-fidelity simulation environment with ManiSkill for rigid body dynamics and Gaussian Splatting (Kerbl et al., 2023) for photorealistic rendering.

Setup. Our hardware platform comprises a Franka Panda robotic arm and an Intel RealSense D435 camera serving as the primary visual sensor. We perform manual calibration by aligning simulated viewpoints with real-world camera perspectives to synchronize their extrinsic matrices. As illustrated in Fig. 1, the photorealistic textures and color profiles in our simulator closely mirror the physical environment, effectively minimizing the visual domain shift from simulation to reality.

Results. Following the experimental protocol established in the ManiSkill benchmarks, we initially collect 20 expert trajectories via a motion planner for few-shot SFT, which is subsequently optimized through RL over 100 training iterations. We deploy the RL fine-tuned policies in the real world in a zero-shot manner. Empirical results indicate that while the SFT baseline fails to complete the task, the RL-tuned policy achieves a 40% success rate. A representative successful episode is visualized in Fig. 12.

F. Ablation Details

F.1. RL algorithms

Given the significant performance gains from PPO on the LIBERO benchmark, we also investigated the effectiveness of GRPO (Shao et al., 2024), another widely used policy gradient method applied in VLA+RL training (Li et al., 2025a). We compare the performance of PPO and GRPO on both the π_0 and $\pi_{0.5}$ models, with results denoted in Tab. 9.

Conclusion. To sum up, our findings highlight a critical

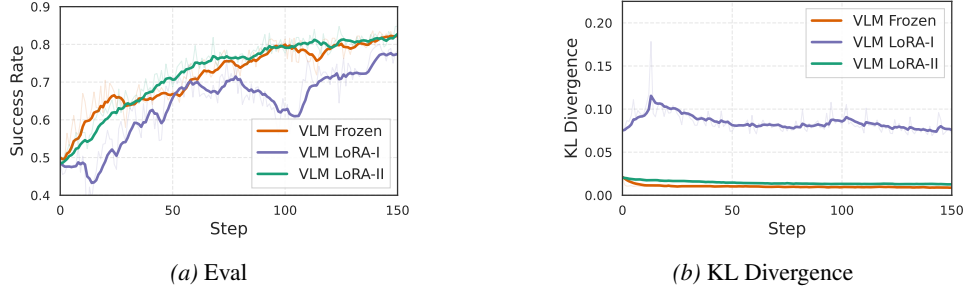


Figure 13. Ablation study on the effectiveness of VLM during RL.

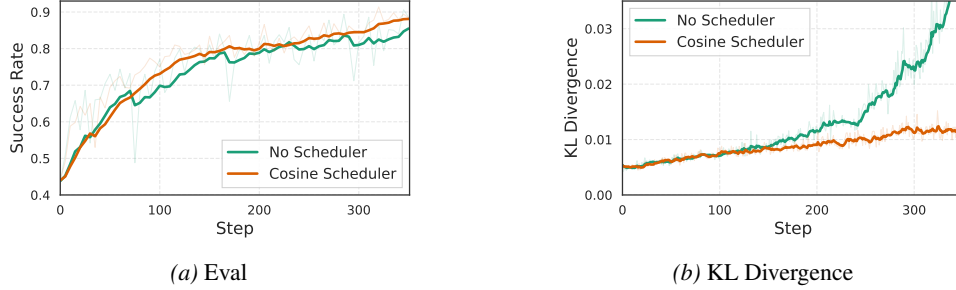


Figure 14. Ablation study on the learning rate scheduler.

trade-off: *parameters tailored for rollout success may adversely impact training stability, ultimately constraining the performance ceiling of RL.* Therefore, careful parameter tuning is required to achieve a synergy between high-quality rollouts and stable policy convergence.

F.2. VLM Fine-tuning Analysis

In our previous experiments, the VLM is frozen, and the optimization is confined exclusively to the action expert during RL. In this subsection, we aim to investigate the role of the VLM during RL. Specifically, we employ Low-Rank Adaptation (LoRA) (Hu et al., 2022) for the VLM, facilitating its joint optimization with the action expert. We set the LoRA rank to $r = 32$ and the scaling parameter to $\alpha = 32$, while the action expert remains fully trainable.

We conduct experiments with the π_0 model with Flow-SDE on the LIBERO-Long benchmark, comparing three distinct configurations:

- **VLM Frozen:** $5e^{-6}$ learning rate, 4 updates/epoch.
- **VLM LoRA-I:** $5e^{-6}$ learning rate, 4 updates/epoch.
- **VLM LoRA-II:** $1e^{-6}$ learning rate, 2 updates/epoch.

As presented in Fig. 13, the VLM LoRA-II configuration achieves a learning trajectory comparable to the VLM frozen baseline. This empirical observation yields two critical inferences: (1) The benefit of fine-tuning the VLM on the

LIBERO benchmark is not evident. We conjecture the limited performance gain owing to the limited scene variability within LIBERO, for which the pretrained VLM representations are already sufficiently robust. (2) Fine-tuning VLM together with the action expert requires a more conservative optimization configuration for training stability.

G. Insights from Large-Scale RL Training

In this section, we elaborate on some empirical insights we gained during RL training.

Hyperparameters. According to the hyperparameters ablation detailed in Sec. 5.4, the performance disparity between the train and eval performance of the initial SFT checkpoint warrants close attention. If this disparity is significant, we recommend either reducing the noise magnitude or increasing the number of denoising steps to mitigate the performance loss when shifting from deterministic to stochastic execution. Furthermore, as previously established, lower noise levels yield larger gradients, requiring a smaller learning rate to maintain training stability.

We also observed that when train performance improves steadily while eval performance oscillates, increasing the number of denoising steps can help alleviate this, benefiting from reduced divergence in the action distributions between the deterministic and stochastic action generation processes. Regarding the action chunk, we empirically found that long-horizon tasks benefit from larger chunk sizes. For instance, we set the chunk size to 10 for LIBERO-Long and 5 for the

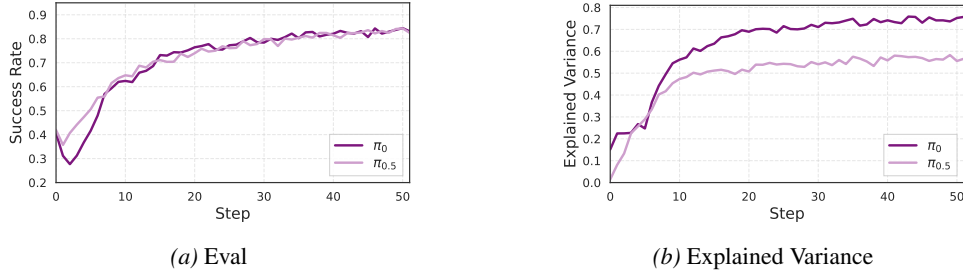


Figure 15. Training curves in ManiSkill.

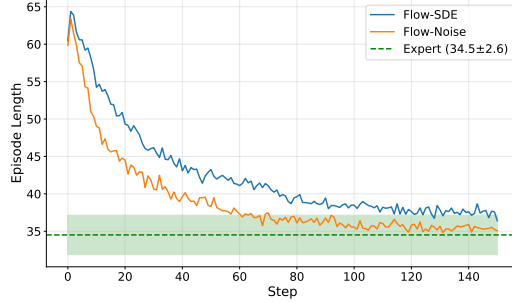


Figure 16. Episode length: $\pi_{0.5}$ RL training in ManiSkill.

other sub-tasks.

Training. In our $\pi_{0.5}$ experiments on the LIBERO-Long benchmark, we observed that the Kullback–Leibler (KL) divergence metric increased steadily throughout training, potentially leading to instability. We mitigated this issue by implementing a learning rate scheduler with cosine annealing. As demonstrated in Fig. 14, this scheduler effectively prevents the KL divergence from escalating, thereby stabilizing the training process.

Critic. In our ManiSkill experiments, we observe that policy evaluation performance exhibits an initial dip before improving for both π_0 and $\pi_{0.5}$ models, as shown in Fig. 15. We attribute this transient degradation to the critic providing inaccurate signals during its warm-up phase. The subsequent eval improvement correlates directly with the critic’s value estimations stabilizing, as evidenced by the rising explained variance.

Temporal Efficiency. We also study how the rollout of RL in a physical simulator helps shape the policy to achieve expert-level temporal efficiency. We analyze the expert motion planning data used for SFT and then tracked the average episode lengths during the RL training of the $\pi_{0.5}$ model in ManiSkill. As shown in Fig. 16, the SFT-initialized policy exhibits significantly longer episodes due to execution errors. In contrast, $\pi_{0.5}$ achieves episode lengths that converge to the expert range after RL training, demonstrating a substantial improvement in temporal efficiency.

We attribute this convergence to two factors: (1) RL en-

hances the policy’s error-correction capabilities, allowing it to recover from execution failures. (2) Our partial reset mechanism incentivizes temporal efficiency through discounted rewards, as faster task completion enables the agent to trigger more resets and accumulate higher total rewards within each update cycle.

H. RL for GR00T N1.5

H.1. Setup

GR00T N1.5. We conduct additional experiments based on the GR00T N1.5 model (Bjorck et al., 2025), which is a foundation model tailored for generalist humanoid robot reasoning and manipulation. The architecture integrates an Eagle 2.5 VLM (Chen et al., 2025), optimized for spatial grounding and physical reasoning, with a Diffusion Transformer head (Peebles & Xie, 2023) for action denoising. It facilitates multi-embodiment compatibility through specialized heads, supporting configurations such as humanoids with dexterous hands or grippers, as well as single-arm manipulators.

Regarding the critic implementation, we estimate value functions across the entire denoising trajectory by integrating the critic network directly with the action head. The complete framework is illustrated in Fig. 17.

Benchmark. We evaluate the model performance of GR00T across four manipulation task suites in LIBERO: *Spatial*, *Object*, *Goal*, and *Long*.

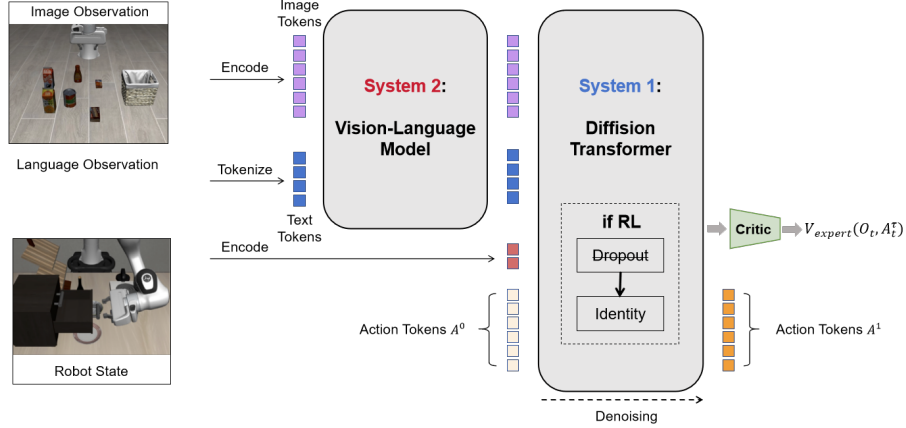


Figure 17. Illustration for the architecture of GR00T-N1.5.

Table 10. Results of Finetuning GR00T using PPO with Flow-SDE on the LIBERO.

Model		LIBERO					
		Spatial	Object	Goal	Long	Avg.	Δ Avg.
GR00T	SFT	41.4	58.6	48.2	61.9	52.5	—
	+PPO	92.5	96.2	84.3	86.6	89.9	+37.4

Implementation Details. Similar to the π_0 implementation, we initiate our process with SFT on expert demonstrations. For the SFT stage, we fine-tune the entire model following the official setting. In the subsequent RL stage, we exclusively fine-tune the action expert model while keeping the vision-language model parameters fixed.

A crucial methodological refinement in our RL pipeline is the replacement of dropout layers in the expert model with identity layers. Dropout is widely recognized to induce instability during online RL training. Specifically, it introduces non-deterministic perturbations to the effective policy, shifting the standard probability ratio update from:

$$\rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (17)$$

to a highly unstable form:

$$\rho_t(\theta) = \frac{\pi_{\alpha_{\text{new}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \quad (18)$$

where $\rho_t(\theta)$ denotes the probability ratio, and α_{new} represents the policy state post-update as modified by the stochastic dropout mask. This structural stochasticity, compounded with per-step policy updates, severely undermines training convergence. The training hyperparameters are identical to those used for π_0 , with Flow-SDE employed as the primary RL algorithm.

H.2. Results

Results are summarized in Tab. 10. For the few-shot model, the SFT baseline achieves only a 52.5% success rate, reflecting limited generalization from sparse demonstrations. Conversely, our RL-based Flow-SDE significantly improves performance to 89.9%. These results, obtained using default π_0 configurations, underscore the broad applicability of our method across architectures. While task-specific tuning could further enhance performance, we leave such optimization for future work.

I. Limitations and Future Work

Noise Injection. Our current noise injection strategy exhibits a performance drop during the ODE-to-SDE conversion. Flow-CPS (Wang & Yu, 2025) attributes this loss to numerical error and proposes an improved coefficients-preserving sampling method. In our experiments, we attempted this configuration. Consistent with our hyperparameter ablation, our experiments showed that while this configuration mitigated the ODE-SDE precision error, it yielded limited RL improvement. Nevertheless, we argue that improving the noise injection strategy holds significant potential, specifically converting the ODE formulation to an SDE formulation while preserving the action distribution undisturbed.

Training Acceleration. Our current implementation of the mixed ODE-SDE rollout is simplistic in Flow-SDE, *i.e.*, it

randomly selects one denoising step as an SDE step, while all other steps remain ODE steps. We posit that future investigations into mixed ODE-SDE rollouts, leveraging advances in accelerating flow-based image generation (Li et al., 2025b; He et al., 2025; Liu et al., 2025b; Li et al., 2025c), could further enhance Flow-SDE, leading to faster training and improved performance.

Generalization. Maniskill OOD tests indicate that the semantic generalization of SFT and RL models remains limited. To address this, future work will leverage RL to enhance robustness by training on more diverse task distributions and varied linguistic instructions, thereby fostering better cross-task adaptability.

J. Training Hyperparameters.

We record the training hyperparameters used to train both π_0 and $\pi_{0.5}$ on each benchmark, and present them in Tabs. 11 to 13.

Table 11. Hyperparameters across LIBERO.

Parameters	Algorithms and Tasks							
	π_0				$\pi_{0.5}$			
	Spatial	Object	Goal	Long	Spatial	Object	Goal	Long
Train epochs	500	500	500	500	500	500	500	500
Global batch size	2048	2048	2048	2048	2048	2048	2048	2048
Update epochs	4	4	4	4	1	1	3	4
Actor lr	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6
Critic lr	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
Scheduler	False	False	False	False	False	False	False	True
Reward discount rate γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95
Clip ratio ϵ	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
Interaction steps	240	240	320	480	240	320	320	480
Parallel environments	64	64	64	64	64	64	64	64
Rollout epochs	8	8	8	8	8	8	8	8
Action prediction horizon H	50	50	50	50	10	10	10	10
Action replan horizon H'	5	5	5	10	5	5	5	10
Denoise steps	4	4	4	4	3	5	5	5
Noise level σ (Flow-SDE)	0.5	0.5	0.5	0.5	0.5	0.3	0.3	0.5
Max log-var (Flow-Noise)	0.16	0.16	0.16	0.16	0.10	0.10	0.10	0.10
Min log-var (Flow-Noise)	0.08	0.08	0.08	0.08	0.04	0.04	0.04	0.04
Entropy bonus (Flow-Noise)	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005

Table 12. Hyperparameters across SIMPLER and ManiSkill.

Parameters	Algorithms and Tasks									
	π_0					$\pi_{0.5}$				
	Eggplant	Carrot	Spoon	Cube	ManiSkill	Eggplant	Carrot	Spoon	Cube	ManiSkill
SFT train steps	1000	1000	1000	1000	1000	1000	1000	1000	1000	1000
RL train steps	40	40	40	130	150	40	40	40	70	150
Global batch size	2560	2560	2560	2560	5120	2560	2560	2560	2560	5120
Update epochs	4	4	4	4	4	4	4	4	4	5
Actor lr	5.6e-6	5.6e-6	5.6e-6	5.6e-6	7.91e-6	5.6e-6	5.6e-6	5.6e-6	5.6e-6	7.91e-6
Critic lr	1.1e-4	1.1e-4	1.1e-4	1.1e-4	1.55e-4	1.1e-4	1.1e-4	1.1e-4	1.1e-4	1.55e-4
Scheduler	False	False	False	False	False	False	False	False	False	False
Reward discount rate γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95
Clip ratio ϵ	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
Interaction steps	48	48	48	48	48	48	48	48	48	48
Parallel environments	256	256	256	256	320	256	256	256	256	320
Rollout epochs	1	1	1	1	1	1	1	1	1	1
Action prediction horizon H	8	8	8	8	8	8	8	8	8	8
Action replan horizon H'	5	5	5	5	5	5	5	5	5	5
Denoise steps	4	4	4	4	4	4	4	4	4	4
Noise level σ (Flow-SDE)	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
Max log-var (Flow-Noise)	0.16	0.16	0.16	0.16	0.16	0.10	0.10	0.10	0.10	0.10
Min log-var (Flow-Noise)	0.08	0.08	0.08	0.08	0.08	0.04	0.04	0.04	0.04	0.04
Entropy bonus (Flow-Noise)	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005

Table 13. Hyperparameters across MetaWorld and CALVIN benchmarks.

Parameters	Benchmarks and models			
	MetaWorld		CALVIN	
	π_0	$\pi_{0.5}$	π_0	$\pi_{0.5}$
Train epochs	450	450	100	100
Global batch size	2048	2048	2048	2048
Update epochs	4	4	4	4
Actor lr	1e-5	5e-6	5e-6	5e-6
Critic lr	1e-4	1e-4	1e-4	1e-4
Scheduler	False	True	False	False
Reward discount rate γ	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95
Clip ratio ϵ	0.2	0.2	0.2	0.2
Interaction steps	100	100	480	480
Parallel environments	64	64	64	64
Rollout epochs	8	8	8	8
Action prediction horizon H	5	5	5	5
Action replan horizon H'	5	5	5	5
Denoise steps	5	5	5	5
Noise level σ (Flow-SDE)	0.5	0.5	0.5	0.5
Max log-var (Flow-Noise)	0.10	0.10	0.16	0.16
Min log-var (Flow-Noise)	0.04	0.04	0.08	0.08
Entropy bonus (Flow-Noise)	0.005	0.005	0.005	0.005